TECHNICAL ARCHITECTURE REFERENCE

Eye-Q

An open-vocabulary vision–language retrofit appliance that turns conventional IP cameras into cameras you can ask questions of, in plain English, with no model training and no cloud.

Revision e57a6dd · branch feat/per-subject-tracking-and-vlm-load-control · 21 August 2026 · 8,793 LOC application / 3,790 LOC tests

§0 Abstract

Conventional IP cameras detect motion; they cannot answer questions. Retrofitting semantic alerting onto them normally means training a bespoke detector for every condition — a hard-hat model, a spill model, a blocked-exit model — putting each new question behind its own labelling cycle. Eye-Q removes that cycle: an operator types the condition as a sentence, *"Is this person bending over?"*, and a vision–language model answers it per subject, per frame, with no training and nothing leaving the device.

The problem this creates is arithmetic. A VLM able to answer an arbitrary question about an image costs *seconds* per query on edge CPUs, while a camera delivers thirty frames a second. Eye-Q resolves the mismatch with a two-stage cascade that arbitrages three orders of magnitude of latency: YOLOv8n runs continuously at 2–4 ms per frame and does nothing but decide when reasoning is worth paying for; Moondream2 wakes only for the subjects that survive that gate. Between them sits a zero-copy shared-memory ring whose readers are lock-free and stale-tolerant, so the slow stage can lag seconds behind live without the fast stage ever blocking.

The substance lies after the gate. Stage 1 assigns every detection a stable identity, so a frame of five workers yields five judgements rather than one — which multiplies inference demand by the size of the crowd and must therefore be paid for. Stage 2 pays with four ordered layers of load control — coalescing superseded triggers, memoising each subject's verdict, prioritising subjects never yet seen, truncating the batch — all governed by one formula an operator can reason about: `duty = subjects × inference_s ÷ ttl`. Correctness measures carry equal weight: each crop is dimmed outside the subject's own box so a neighbour cannot flip its verdict, and every crop is taken *before* the first slow inference, so inference latency can never outlive the buffer it depends on.

Purpose	Plain-English alert rules over existing RTSP / USB / file sources; no per-condition training
Topology	One Python 3.13 process — four daemon threads plus an asyncio control plane
Two stages	YOLOv8n at ~2–4 ms/frame gates Moondream2 1.86 B at ~5.6 s/query (measured, Apple-Silicon CPU)
Transport	ZeroMQ PUB/SUB, two endpoints — `:5557` detections, `:5558` verdicts
Frame store	POSIX shared-memory ring — 150 slots, 414.7 MB at 1280×720, lock-free readers
Control plane	FastAPI + uvicorn, 17 routes, MJPEG preview, WebSocket broadcast; SQLite in WAL mode
Cloud dependency	None — all three VLM backends execute locally
Test suite	268 collected; 265 pass, 3 deselected as slow (23.1 s)

> **Scope and method.** Written against a complete read of the source at `e57a6dd`, with every file reference re-verified against the working tree. The seven diagram subjects originally scoped are consolidated into four figures to fit this format — deployment topology is carried by the process boundary in Figure 1, module and data-flow views fold into Figure 2, and the component inventory is given as tables. Claims the source could not substantiate are omitted; see the colophon on page 4.

§1 System architecture

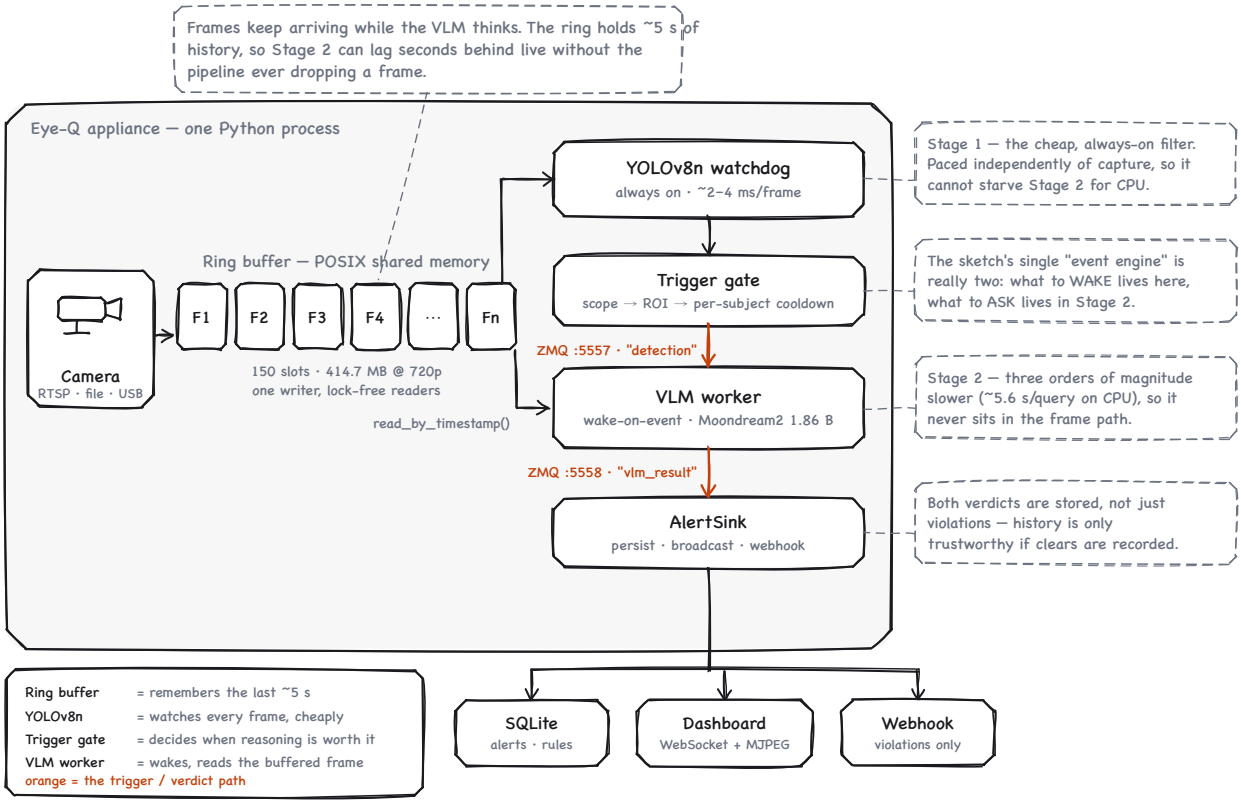


Figure 1 — System architecture. One OS process. Capture writes into a shared-memory ring; Stage 1 reads the newest frame and decides only whether reasoning is warranted; Stage 2 wakes on that decision and fetches the pixels back by timestamp. The gate is drawn twice because *what to wake* and *what to ask* are separate concerns.

1.1 One process, four unequal threads

Everything runs in one process because of the ring buffer: a shared-memory frame store removes copies only if consumers reach it without serialising, and keeping both stages as threads in the owning process makes that free. The segments are nonetheless *named* POSIX objects, so the multi-process seam exists when wanted. The threads are deliberately unequal — capture writes into a ring that overwrites rather than fills, Stage 1 is paced independently of capture because otherwise it starves the stage it exists to feed, and Stage 2 is kept entirely off the frame path.

THREAD	OWNS	CADENCE	UNDER FAILURE
uvicorn loop	Routes, `AlertSink`, WebSocket fan-out	request-driven	A malformed verdict is logged; the sink survives. JPEG encoding runs off-thread.
VideoCapture	`cv2.VideoCapture`, ring writes	`target_fps` or the file's own FPS	Backoff 1 → 30 s over five attempts; files loop seamlessly.
YOLOWatchdog	Model, tracker, cooldown gate, overlay ring	`watchdog_fps`, apart from capture	Warm-up failure falls back to CPU; repeated frames skipped by timestamp.
VLMWorker	Backend, verdict memo, both its ZMQ sockets	trigger-driven, 1 s idle poll	A blown timeout abandons the batch — one executor worker, so a hang blocks it.
VLMInfer pool	One in-flight inference	serial by design	`future.result(timeout)` bounds the query and fails fast.

1.2 Two transport planes, single ownership

Detections travel on `:5557`, verdicts return on `:5558`. The split is not tidiness — ZeroMQ sockets are not thread-safe, so each publisher owns its socket in the thread that created it, and Stage 2 builds both of its own inside its worker loop. Sends are non-blocking behind a high-water mark: a slow consumer drops messages rather than stalling a producer. Ownership is likewise singular — the FastAPI lifespan builds every component in dependency order and tears it down in reverse, the fix for a defect where split ownership let the lifespan overwrite already-wired components and silently sever rule delivery to the VLM.

§2 End-to-end workflow and data flow

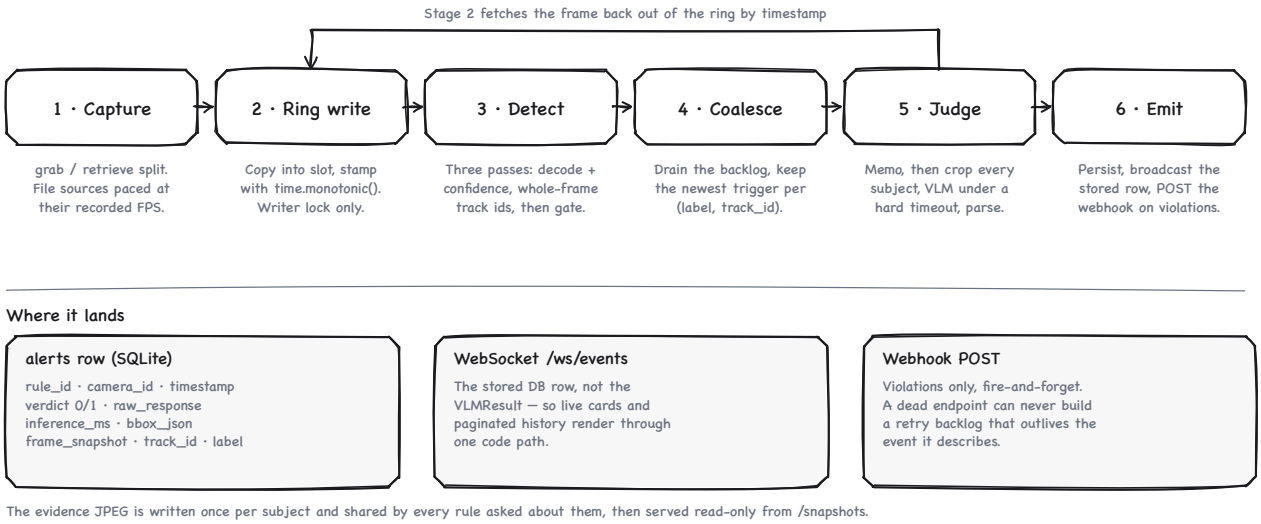


Figure 2 — From frame to alert. Six stages and the three places a result lands. The return path is load-bearing: Stage 2 fetches the frame back out of the ring rather than carrying pixels forward, so the crop must finish before the ring wraps.

Two invariants govern this path, each found by fixing the failure its absence causes. **Crop before inference:** inference takes seconds and the ring holds seconds, so fetching a frame just before each query leaves the second subject in a group with its frame already overwritten — invisible in a one-subject scene, total in a crowded one. **Coalesce by subject, not label:** keeping only the newest trigger per class discards four of five workers as "superseded" when they are different people. Rules flow the other way: `sync_rules()` hands Stage 1 the *coarsest* filter satisfying every rule.

§3 The AI/ML layer

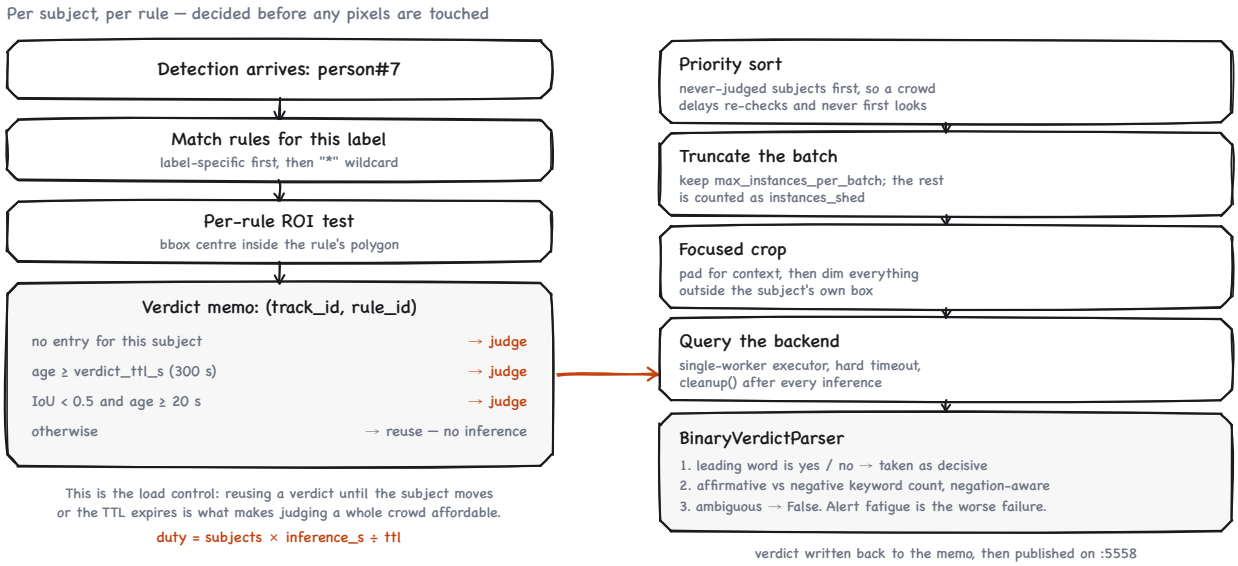


Figure 3 — The Stage 2 decision path. The left half decides whether a question is worth asking and runs before any pixels are touched; the memo there is the load control. Two contracts bind the language surface: "Yes" means violation, so prompts must be phrased that way round, and each rule asks a single predicate.

Stage 1 is the motion filter, not the classifier: YOLOv8n can only ever say *"person"* about a warehouse worker, and every question worth asking is asked by Stage 2 in English. Identity must be resolved before the gate can work, and is — a greedy IoU tracker matches boxes within a label across frames, chosen over ByteTrack because Ultralytics resolves its `lap` dependency at import time by attempting a runtime `pip install`, a hard failure offline. A swap under occlusion costs one redundant inference, never a missed alert.

§4 Innovation — what is genuinely hard here

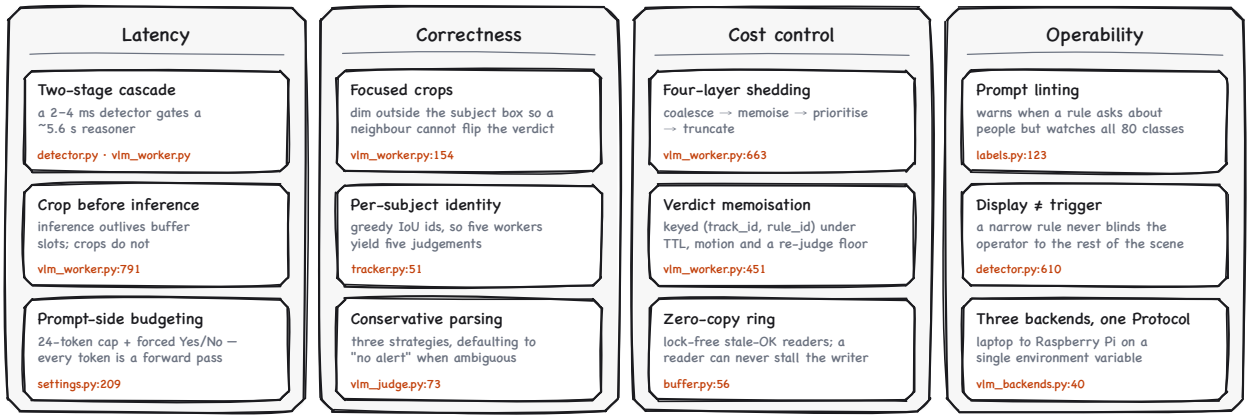


Figure 4 — The twelve strongest ideas in the codebase, grouped by the kind of win each buys and tagged with its source. Every one is running code at this revision.

Two carry disproportionate weight. **Four-layer load control with a published formula** is what makes per-subject judging affordable rather than merely correct, and the status endpoint exposes `instances_judged`, `instances_memoised` and `instances_shed` so the arithmetic can be checked against reality. **Focused crops** are the subtlest: the padding that lets a VLM answer relational questions at all is the same padding that drags in the neighbours.

§5 Deployment and security posture

CONCERN	POSITION AT THIS REVISION
Runtime	`uv sync`, Python ≥ 3.13, one uvicorn process; weights fetched on first run. The device is probed MPS → CUDA → CPU *functionally* rather than by capability flag, because the flag lies, and the VLM defaults to CPU on measurement: 5.6 s against 6.1 s on MPS.
VLM backends	Three behind one `Protocol`, one variable apart: `moondream` in-process (default), `transformers` interruptible, `openai_compat` to a local llama.cpp / Ollama server.
Edge profile	`.env.pi5` targets an 8 GB ARM board: `openai_compat`, 960×540, watchdog 4 FPS, 900 s TTL, batch 3 — ~3 GB of 8. **Unverified on hardware.**
Network posture	No egress by default; the opt-in webhook is the only outbound path. **No authentication** — CORS ∗, bind `0.0.0.0:8000`, every mutating endpoint open. Trusted-LAN only.

§6 Limitations

GAP	SEVERITY	CONSEQUENCE
No authentication or TLS	High	Anyone on the network can watch the feed, rewrite rules and read evidence.
No data retention policy	High	`alerts` rows and snapshot JPEGs grow unbounded; only in-memory state is pruned.
Single camera; dashboard on public CDNs	Medium	One capture though the schema is multi-camera; Tailwind and fonts load remotely, so the UI is not air-gap clean.
No motion model; fire-and-forget webhooks; no ROI editor; dead surface	Low	~2 ids per person at 6 FPS — redundant inferences, never missed alerts. No webhook retry; `roi_json` undrawable; `SystemStatus` and `parse_with_confidence()` unused.

Colophon and method. Written against a full read of the source at `e57a6dd`, with every file reference re-verified against the working tree; the test figure is the actual output of `pytest -q -m "not slow"` — 265 passed, 3 deselected, 23.1 s. Figures are hand-authored vector SVG, generated by `docs/diagrams/build.py` and inlined. **Claims omitted:** the README states Stage 1 "filters 99%+ of frames" and that VLM utilisation drops "from 100% to <1%"; neither is measured anywhere in the repository, so neither appears here — §4's duty-cycle arithmetic is what the code implements, and where the README rounds the ring to "~405 MB" this uses the computed 414.7 MB. Two drifts noted: `demo_cctv.py` carries an uncommitted change contradicting its own docstring's filename, and the pytest cache still lists a removed test file — hence 268 collected, not 311.